

Smytten E-Commerce Database Management System

CHAPTER – 5

PRODUCT AND CATEGORY MANAGEMENT SYSTEM

Introduction

The Smytten E-Commerce Database Management System is designed to manage product and category information efficiently. The system uses two related tables, Category and Product, to store and organize product details. Primary key and foreign key constraints are used to maintain data integrity. SQL operations such as insertion, updating, and deletion are performed on the Product table. A category-wise product report is generated using SQL and PL/SQL to analyze the products available under each category.

2. Objectives

- Design tables for Product and Category entities.
- Define primary and foreign key relationships.
- Store product name, category, price and stock.
- Perform product insertion, updating and deletion.
- Generate category-wise product reports using SQL and PL/SQL.

3. Table Design

3.1 Category2 Table

Attribute	Data Type	Key	Description
Category_ID	NUMBER	Primary Key	Unique category identifier
Category_Name	VARCHAR2(100)	-	Product category name

3.2 Product2 Table

Attribute	Data Type	Key	Description
Product_ID	NUMBER	Primary Key	Unique product identifier
Product_Name	VARCHAR2(100)	-	Product name
Price	NUMBER (10,2)	-	Product price
Stock	NUMBER	-	Available quantity
Category_ID	NUMBER	Foreign Key	References Category2(Category_ID)

4. Relationship

Category and Product have a one-to-many (1:M) relationship. One category can contain many products, while each product belongs to one category.

CATEGORY (1) ————— (M) PRODUCT
Category_ID (PK) Category_ID (FK).

5. SQL Table Creation

```
SQL> CREATE TABLE Category2 (  
2     Category_ID NUMBER PRIMARY KEY,  
3     Category_Name VARCHAR2(100) NOT NULL UNIQUE  
4 );
```

Table created.

```
SQL> CREATE TABLE Product2 (  
2     Product_ID NUMBER PRIMARY KEY,  
3     Product_Name VARCHAR2(100) NOT NULL,  
4     Price NUMBER(10,2) NOT NULL CHECK (Price >= 0),  
5     Stock NUMBER NOT NULL CHECK (Stock >= 0),  
6     Category_ID NUMBER NOT NULL,  
7     CONSTRAINT FK_Product_Category  
8     FOREIGN KEY (Category_ID)  
9     REFERENCES Category2(Category_ID)  
10 );
```

Table created.

6. Insert Records

```
SQL> INSERT INTO Category2  
2     VALUES (1, 'Skincare');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Category2  
2     VALUES (2, 'Makeup');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Category2  
2     VALUES (3, 'Haircare');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Category2  
2     VALUES (4, 'Wellness');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Category2  
2     VALUES (5, 'Fragrance');  
  
1 row created.  
  
SQL>  
SQL> COMMIT;
```

```

SQL> INSERT INTO Product2
  2 VALUES (101, 'Face Wash', 299.00, 50, 1);

1 row created.

SQL>
SQL> INSERT INTO Product2
  2 VALUES (102, 'Moisturizer', 399.00, 40, 1);

1 row created.

SQL>
SQL> INSERT INTO Product2
  2 VALUES (103, 'Lipstick', 499.00, 35, 2);

1 row created.

SQL>
SQL> INSERT INTO Product2
  2 VALUES (104, 'Mascara', 450.00, 30, 2);

1 row created.

SQL>
SQL> INSERT INTO Product2
  2 VALUES (105, 'Shampoo', 349.00, 45, 3);

1 row created.

SQL>
SQL> INSERT INTO Product2
  2 VALUES (106, 'Hair Serum', 599.00, 25, 3);

```

```

SQL> INSERT INTO Product2
  2 VALUES (107, 'Vitamin Gummies', 699.00, 20, 4);

1 row created.

SQL>
SQL> INSERT INTO Product2
  2 VALUES (108, 'Body Mist', 550.00, 28, 5);

1 row created.

SQL>
SQL> INSERT INTO Product2
  2 VALUES (109, 'Face Serum', 799.00, 18, 1);

1 row created.

SQL>
SQL> INSERT INTO Product2
  2 VALUES (110, 'Conditioner', 379.00, 32, 3);

1 row created.

SQL>
SQL> COMMIT;

Commit complete.

```

7. Product Insertion, Updating and Deletion

```

SQL> UPDATE Product2
  2 SET Price = 329.00
  3 WHERE Product_ID = 101;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.

SQL> UPDATE Product2
  2 SET Stock = 35
  3 WHERE Product_ID = 106;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.

```

```
SQL> SELECT * FROM Product2
2 WHERE Product_ID IN (101, 106);
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
PRICE      STOCK CATEGORY_ID
-----
101
Face Wash   329        50          1
106
Hair Serum  599        35          3

PRODUCT_ID
-----
PRODUCT_NAME
-----
PRICE      STOCK CATEGORY_ID
-----
```

```
SQL> DELETE FROM Product2
2 WHERE Product_ID = 108;
```

1 row deleted.

```
SQL>
SQL> COMMIT;
```

Commit complete.

```
SQL> DELETE FROM Product2
2 WHERE Product_ID = 107;
```

1 row deleted.

```
SQL>
SQL> COMMIT;
```

Commit complete.

```
SQL> SELECT * FROM Product2;
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
PRICE      STOCK CATEGORY_ID
-----
101
Face Wash   329        50          1
102
Moisturizer 399        40          1

PRODUCT_ID
-----
PRODUCT_NAME
-----
PRICE      STOCK CATEGORY_ID
-----
103
Lipstick    499        35          2
104
Mascara
```

8. Category-wise Product Report (SQL)

```
SQL> SELECT
  2     c.Category_Name,
  3     COUNT(p.Product_ID) AS Total_Products,
  4     NVL(SUM(p.Stock), 0) AS Total_Stock
  5 FROM Category2 c
  6 LEFT JOIN Product2 p
  7     ON c.Category_ID = p.Category_ID
  8 GROUP BY c.Category_Name
  9 ORDER BY c.Category_Name;
```

CATEGORY_NAME	TOTAL_PRODUCTS	TOTAL_STOCK
Fragrance	0	0
Haircare	3	112
Makeup	2	65

9. PL/SQL Program – Category-wise Report

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2
  3     CURSOR Category_Report IS
  4     SELECT
  5         c.Category_Name,
  6         COUNT(p.Product_ID) AS Total_Products,
  7         NVL(SUM(p.Stock), 0) AS Total_Stock
  8     FROM Category2 c
  9     LEFT JOIN Product2 p
 10         ON c.Category_ID = p.Category_ID
 11     GROUP BY c.Category_Name
 12     ORDER BY c.Category_Name;
 13
 14 BEGIN
 15     DBMS_OUTPUT.PUT_LINE('=====');
 16     DBMS_OUTPUT.PUT_LINE('  CATEGORY-WISE PRODUCT REPORT');
 17     DBMS_OUTPUT.PUT_LINE('=====');
 18
 19     FOR rec IN Category_Report LOOP
 20
 21         DBMS_OUTPUT.PUT_LINE(
 22             'Category: ' || rec.Category_Name ||
 23             ' | Products: ' || rec.Total_Products ||
 24             ' | Stock: ' || rec.Total_Stock
 25         );
 26
 27     END LOOP;
 28
 29     DBMS_OUTPUT.PUT_LINE('=====');
 30     DBMS_OUTPUT.PUT_LINE('Report generated successfully.');
```

10. PL/SQL Program – Product Price Update

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2     v_product_id Product2.Product_ID%TYPE := 101;
  3     v_old_price Product2.Price%TYPE;
  4     v_new_price Product2.Price%TYPE := 349;
  5 BEGIN
  6     SELECT Price INTO v_old_price
  7     FROM Product2
  8     WHERE Product_ID = v_product_id;
  9
 10     UPDATE Product2
 11     SET Price = v_new_price
 12     WHERE Product_ID = v_product_id;
 13
 14     COMMIT;
 15
 16     DBMS_OUTPUT.PUT_LINE('Product ID: ' || v_product_id);
 17     DBMS_OUTPUT.PUT_LINE('Old Price: ' || v_old_price);
 18     DBMS_OUTPUT.PUT_LINE('New Price: ' || v_new_price);
 19     DBMS_OUTPUT.PUT_LINE('Product price updated successfully.');
```

```
20
21 EXCEPTION
22 WHEN NO_DATA_FOUND THEN
23     DBMS_OUTPUT.PUT_LINE('Product not found.');
```

```
24 END;
25 /
Product ID: 101
Old Price: 329
New Price: 349
Product price updated successfully.
```

11. Expected Category-wise Report

Category	Total Products	Total Stock
Fragrance	1	28
Haircare	3	102
Makeup	2	65
Skincare	3	108
Wellness	1	20

12. Conclusion

The Product and Category database design successfully represents the one-to-many relationship between categories and products. Primary and foreign key constraints maintain data integrity, while INSERT, UPDATE and DELETE operations support product management. The category-wise report provides a useful summary of products and stock. The PL/SQL cursor program demonstrates procedural database programming for report generation.